Solution: Container With Most Water

Problem Statement

Given an array of non-negative integers, where each integer represents the `height` of a vertical line positioned at index `i`. You need to find the two lines that, when combined with the x-axis, form a container that can hold the most water.

The goal is to find the maximum amount of water (area) that this container can hold.

Note: The water container's width is the distance between the two lines, and its height is determined by the shorter of the two lines.

Examples

Example 1:

- **Input**: [1,3,2,4,5]
- **Expected Output**: 9
- **Justification**: The lines at index 1 and 4 form the container with the most water. The width is 3 * (4-1), and the height is determined by the shorter line, which is 3. Thus, the area is 3 * 3 = 9.

Example 2:

- **Input**: [5,2,4,2,6,3]
- **Expected Output**: 20
- **Justification**: The lines at index 0 and 4 form the container with the most water. The width is 5 * (4-0), and the height is determined by the shorter line, which is 5. Thus, the area is 5 * 4 = 20.

Example 3:

- **Input**: [2,3,4,5,18,17,6]
- **Expected Output**: 17
- **Justification**: The lines at index 4 and 5 form the container with the most water. The width is 17 * (5-4), and the height is determined by the shorter line, which is 17. Thus, the area is 17 * 1 = 17.

Solution

The "Container With Most Water" problem can be efficiently solved using a two-pointer approach. The essence of the solution lies in the observation that the container's capacity is determined by the length of the two lines and the distance between them.

By starting with two pointers at the extreme ends of the array and moving them toward each other, we can explore all possible container sizes. At each step, we calculate the area and update our maximum if the current area is larger. The key insight is to always move the pointer pointing to the shorter line, as this has the potential to increase the container's height and, thus, its capacity.

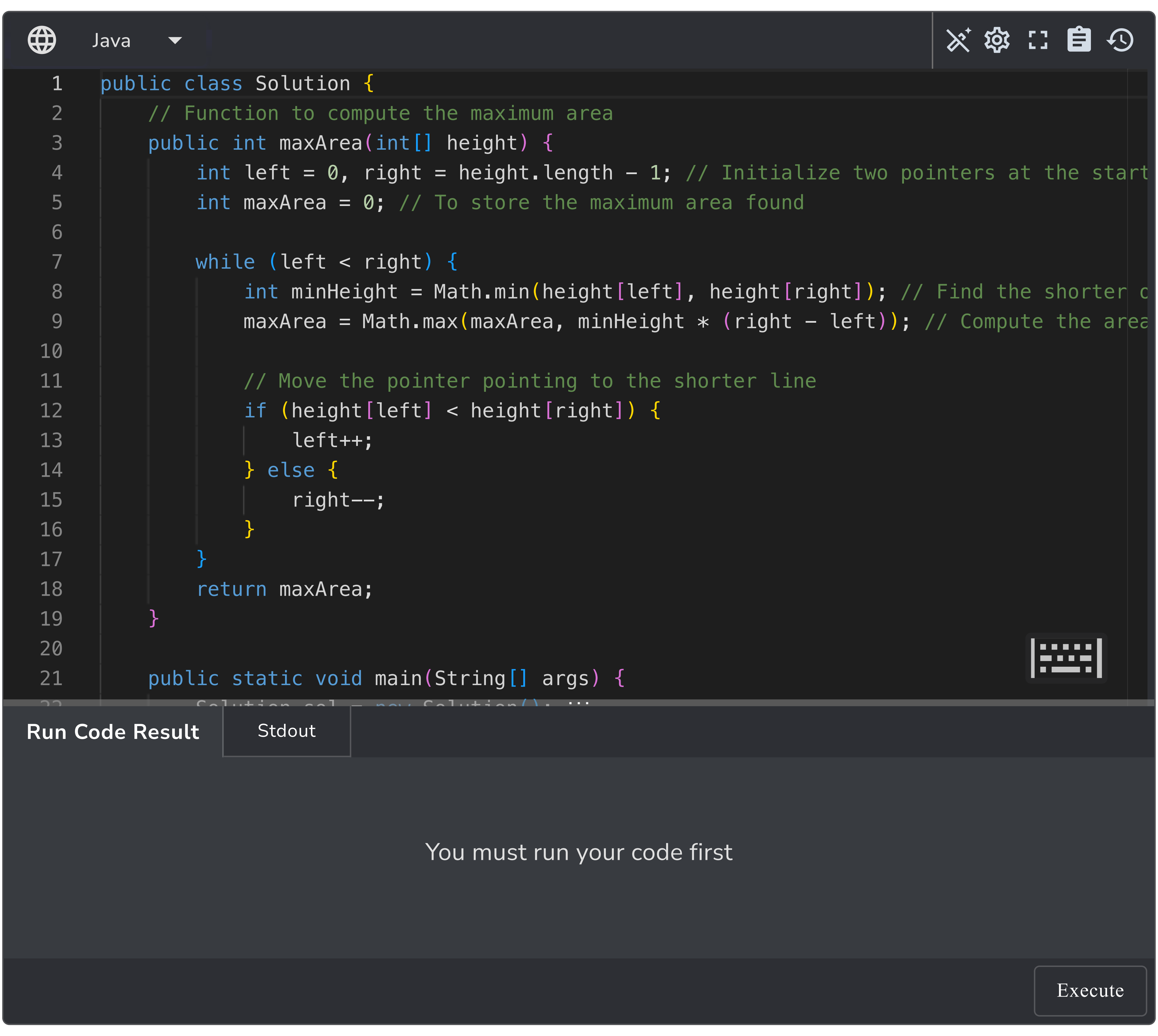
1. **Initialization**: Begin by initializing two pointers, one at the start (`left`) and one at the end (`right`) of the array. Also, initialize a variable `maxArea` to store the maximum area found.
2. **Pointer Movement**: At each step, calculate the area formed by the lines at the `left` and `right` pointers. If this area is greater than `maxArea`, update `maxArea`. Then, move the pointer pointing to the shorter line towards the other pointer. This is because by moving the taller line, we might miss out on a larger area, but by moving the shorter line, we have a chance of finding a taller line and, thus a larger area.
3. **Termination**: Continue moving the pointers until they meet. At this point, we have considered all possible pairs of lines.
4. **Return**: Once the pointers meet, return the `maxArea`.

Algorithm Walkthrough

Using the input [1,3,2,4,5]:

- Initialize `left` to 0 and `right` to 4. `maxArea` is 0.
- Calculate area with `left` and `right`: min(1,5) * (4-0) = 4. Update `maxArea` to 4.
- Move the `left` pointer to 1 since height at `left` is shorter.
- Calculate area with `left` and `right`: min(3,5) * (4-1) = 9. Update `maxArea` to 9.
- Move the `left` pointer to 2.
- Calculate area with `left` and `right`: min(2,5) * (4-2) = 4. `maxArea` remains 9.
- Move the `left` pointer to 3.
- Calculate area with `left` and `right`: min(4,5) * (4-3) = 4. `maxArea` remains 9.
- Pointers meet. Return `maxArea` which is 9.

Code



```java
public class Solution {
    // Function to compute the maximum area
    public int maxArea(int[] height) {
        int left = 0, right = height.length - 1; // Initialize two pointers at the start
        int maxArea = 0; // To store the maximum area found

        while (left < right) {
            int minHeight = Math.min(height[left], height[right]); // Find the shorter c
            maxArea = Math.max(maxArea, minHeight * (right - left)); // Compute the area

            // Move the pointer pointing to the shorter line
            if (height[left] < height[right]) {
                left++;
            } else {
                right--;
            }
        }
        return maxArea;
    }

    public static void main(String[] args) {
```

You must run your code first

Complexity Analysis:

- **Time Complexity**: O(n) - We traverse the array once using two pointers.
- **Space Complexity**: O(1) - We use a constant amount of space.

← Previous
Container With Most Water (Medium)

Next →
Palindromic Substrings (Medium)

☐ Mark as Completed